PROO/SEW 3 - Assignment: *Billa Plus Plus Plus*

Objective

Create a program for managing a supermarket's articles.

Things To Learn

- Working with *hash maps*.
- Working with *enums*.
- Applying the *Abstract Factory*-pattern.

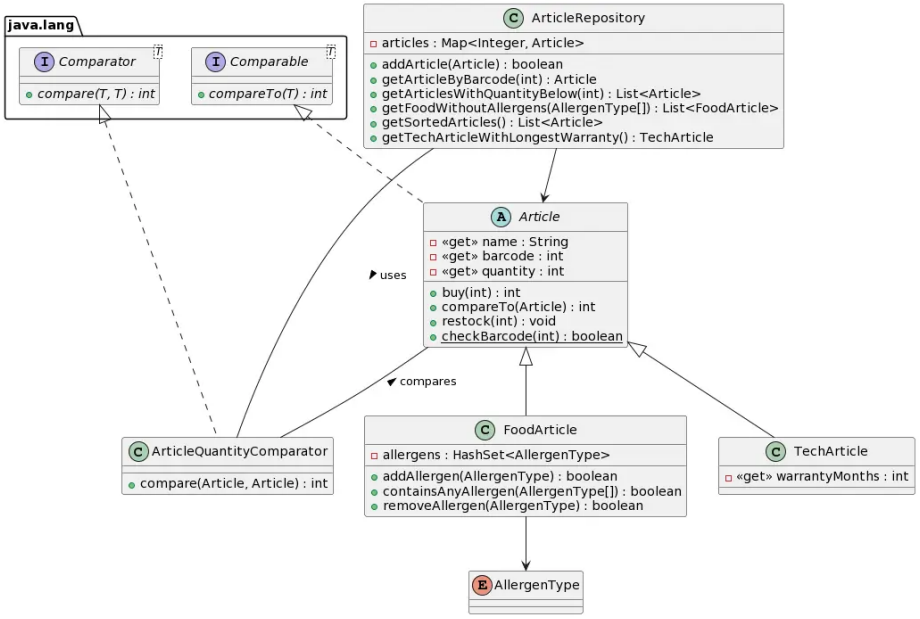
Submission Guidelines

- Your implemented solution as ***zipped*** *IntelliJ*-project.

Task

You are tasked with implementing the article management for the new supermarket *Billa Plus Plus* (or *B++* for short). Right now, the product range solely consists of *tech gadgets* and food items - but we need to keep in mind, that the assortment will surely grow in the near future!

Take a look at the following diagram and the implementation details below:



1. `Article`

- The `abstract` base class for all articles the market sells.
- Negative article quantities should be corrected to `0`.
- The quantity can be changed using the `buy` and `restock` methods, both throwing an `IllegalArgumentException` if a negative value is passed. Additionally, the `buy` method should return the quantity that was *actually* removed from stock.
- Implements the `Comparable`-interface using the article name for ordering.
- Provides a `static`-method for checking the validity of a barcode. See section below for details.
- Invalid barcodes passed to the constructor should lead to an `IllegalArgumentException`.
- A simple `toString`-method should provide a neat *String*-representation of an article.

2. `FoodArticle`
   - The class for all groceries, inheriting from `Article`.
   - Additionally stores the food's allergens in a `HashSet` using the ***enum****erated type* `AllergenType`.
     - Currently there are 14 food ingredients that must be declared as allergens: `A`, `B`, `C`, `D`, `E`, `F`, `G`, `H`, `L`, `M`, `N`, `O`, `P`, `R`.
   - The constructor accepts an array of allergens. The `HashSet`'s `addAll`-method could prove to be useful for this.
   - The class provides some methods for managing a food item's allergens. Both `addAllergen` and `removeAllergen` should return the *success* of the respective operation - an allergen can't be added twice, a non-existing allergen can't be removed - while `containsAnyAllergen` should return `true` if the food item has *any* of the passed allergens.
3. `TechArticle`
   - The class for all *tech gadgets*, inherits from `Article` and extends its properties by *warranty months*.
   - By law, *tech gadgets* are required to provide half a year of warranty. Invalid values should automatically be corrected.
4. `ArticleRepository`
   - Stores the `Article`s in a *hash map* using their barcodes as keys.
   - Provides various methods for adding and accessing articles:
     - `addArticle` adds an article to the map and returns `true` if successful. Articles need to have a unique barcode and a *positive* quantity!
     - `getArticleByBarcode` returns an article by the barcode passed.
     - `getArticlesWithQuantityBelow` returns a list of articles with the stock below a passed value. The articles should be sorted by their quantity. This can be achieved by implementing the `ArticleQuantityComparator`.
     - `getFoodWithoutAllergens` returns a list of articles (sorted by their names) not containing **either** of the passed allergens.
     - `getSortedArticles` returns a list of articles sorted by their

names.
- `getTechArticleWithLongestWarranty` returns the gadget with the longest warranty. Returns `null` if there are no `TechArticle`s in the store.

EAN-8 Validation

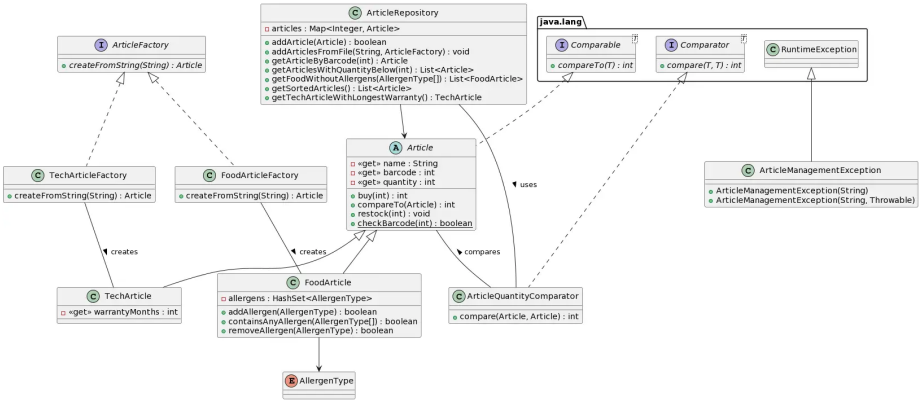
The *EAN-8* barcode is used for small packages such as chewing gums or cigarettes - or in our case small data types like integers. This globally unique item number shares the same validation algorithm as its *big brothers* having 12, 13 and 14 digits.

The following diagram demonstrates the algorithm for the barcode *4534232X*:

Number	4	5	3	4	2	3	2	
Weight	1	3	1	3	1	3	1	
Products	4	15	3	12	2	9	2	
Sum								47
Sum *mod* 10								7
Difference to next 10								**3**

Thus, the complete valid barcode is *45342323*.

Extension



After another *rebranding*, *Billa Plus Plus* ***Plus*** wants to store their products in `.csv`-files and tasks you with extending the article management so that it can load articles from files.

Since you know, that the product range will be expanded in the near future, you decide to apply the *Abstract Factory*-pattern. This way, you'll be able to extend the articles without modifying the existing code (aka *Open-Closed-Principle*).

For this, create an *interface* `ArticleFactory` and create two concrete factories for both article types implementing the `createFromString`-method.

Extend your `ArticleRepository` by an `addArticlesFromFile`-method, that accepts both the file path and the factory implementation that is used for parsing the lines in the `.csv`-file.

A simple *unchecked* `ArticleManagementException` should be used for handling problems when parsing or loading the files.